Metadata Management & Multi-Format Document Inspector

1. Central Concept

The main goal of this lesson is:

Organise LangChain Document objects coming from different sources using consistent metadata, and verify that the loaded content is usable.

Master flow:

TXT + PDF + CSV + HTML

↓

LangChain Documents

↓

Metadata Standardisation

↓

Document Inspector

Think of this like a:

Universal Adapter

Different source formats come in, but we organise them into one consistent structure.

2. Why Is Metadata Important?

For an LLM, content alone is not always enough.

Example:

Employees receive 12 casual leaves.

The content tells us the information.

But metadata can tell us:

Which file?

Which department?

Which source type?

Which page?

Which row?

So:

Content

↓

What is the information?

Metadata

↓

Where did the information come from?

Important learning:

Content gives meaning. Metadata gives context.

3. What Is Metadata Standardisation?

Definition

Metadata standardisation means maintaining a consistent set of descriptive fields across Documents from different sources.

Suppose we have three Documents.

Text Document

```
{
   "source":
      "library_rules.txt"
}
```

PDF Document

```
{
   "source":
      "company_policy.pdf",

   "page":
      0,
```

```
  "total_pages":
    3
}
```

CSV Document

```
{
  "source":
    "students.csv",

  "row":
    0
}
```

These Documents have different source-specific metadata.

But we may want some common fields in all Documents.

For example:

source

source_type

department

category

So:

Different Sources

↓

Different Original Metadata

↓

Add Common Metadata

↓

Consistent Structure

4. Why Standardise Metadata?

Metadata standardisation makes Documents easier to:

- Organise
- Inspect
- Filter
- Process consistently
- Identify
- Group

Simple:

Different Documents

↓

Common Metadata Structure

↓

Easy Processing

5. Real-Life Example — Hospital Records

Imagine a hospital has records from:

Cardiology

Orthopaedics

Neurology

Each department may have specialised information.

But all records may still require common fields such as:

Patient ID

Department

Record Type

Date

Important:

Standardisation does not mean removing specialised information.

Instead:

Preserve Special Information

+

Add Common Information

↓

Standardised Record

The same principle applies to LangChain Documents.

6. Updating Metadata Without Losing Original Information

Suppose:

```
document.metadata = {
   "source":
      "company_policy.pdf",

   "page":
      0
}
```

Now we want to add:

department = HR

If we replace the whole dictionary carelessly, we may lose:

source

page

Instead, we should enrich the existing metadata.

7. Add One Metadata Field

```
document.metadata[
   "department"
] = "HR"
```

This adds a field without removing existing information.

8. Add Multiple Metadata Fields

```
document.metadata.update({
   "department":
      "HR",

   "category":
      "Employee Policy"
})
```

So:

Existing Metadata

\+

New Metadata

↓

Enriched Metadata

Main rule:

Preserve original metadata while adding application-specific information.

9. Program 1 — Enriching Document Metadata

Objective

Add new metadata fields without losing the original source information.

```
from langchain_core.documents import (
    Document
)



# ========================================
# CREATE DOCUMENT
# ========================================

document = Document(
    page_content=
        "Library closes at 6 PM.",
```

```
    metadata={
        "source":
            "library_rules.txt"
    }
)



# ========================================
# PRESERVE ORIGINAL METADATA
# ========================================

original_metadata = (
    document.metadata.copy()
)



# ========================================
# ADD CUSTOM METADATA
# ========================================
```

```
document.metadata.update({
    "source_type":
        "txt",

    "department":
        "Library",

    "category":
        "Notice"
})



# ========================================
# DISPLAY RESULTS
# ========================================

print(
    "SOURCE:",
    document.metadata["source"]
)
```

```
print(
    "SOURCE TYPE:",
    document.metadata["source_type"]
)


print(
    "DEPARTMENT:",
    document.metadata["department"]
)


print(
    "CATEGORY:",
    document.metadata["category"]
)
```

```
print(
    "SOURCE PRESERVED:",

    document.metadata["source"]
    ==
    original_metadata["source"]
)
```

10. Understanding .copy()

```
original_metadata = (
    document.metadata.copy()
)
```

This creates a separate copy of the current metadata.

Purpose:

Current Metadata

↓

copy()

↓

Backup Copy

Then after updating the metadata, we can verify that the original source information is still preserved.

11. Program 1 Expected Result

Conceptually:

SOURCE:

library_rules.txt

SOURCE TYPE:

txt

DEPARTMENT:

Library

CATEGORY:

Notice

SOURCE PRESERVED:

True

Conclusion:

We can enrich LangChain Document metadata without losing the original source information.

12. Program 1 Flow

Document

↓

Original Metadata

↓

copy()

↓

Backup

↓

metadata.update()

↓

Add New Fields

↓

Verify Original Source

13. Program 2 — Handling Zero-Based Page and Row Metadata

Different source types may use indexes.

For example:

PDF

↓

page = 0

CSV

↓

row = 0

These are zero-based indexes.

But for users, we normally display:

Page 1

Row 1

14. Sample Documents

```
from langchain_core.documents import (
    Document
)



documents = [

    Document(
        page_content=
            "Employee leave policy.",

        metadata={
            "source":
                "policy.pdf",

            "page":
                0
        }
    ),
```

```
Document(
    page_content=
        "name: Ravi",

    metadata={
        "source":
            "students.csv",

        "row":
            0
    }
),

Document(
    page_content=
        "Library opens at 8 AM.",

    metadata={
        "source":
```

```
            "library.txt"
        }
    )

]
```

15. Inspecting Page and Row Metadata

```
for document in documents:

    metadata = (
        document.metadata
    )



    print(
        "\nSOURCE:",
        metadata["source"]
    )
```

```
page_index = (
    metadata.get("page")
)


row_index = (
    metadata.get("row")
)


if page_index is not None:

    print(
        "DISPLAY PAGE:",
        page_index + 1
    )


elif row_index is not None:
```

```
        print(
            "DISPLAY ROW:",
            row_index + 1
        )


    else:

        print(
            "PAGE/ROW:",
            "Not applicable"
        )
```

16. Why Use .get()?

We use:

```
metadata.get(
    "page"
)
```

instead of:

```
metadata["page"]
```

because not every Document has page metadata.

Similarly:

metadata.get(

"row"

)

may return:

None

if the field is absent.

17. Zero-Based Index vs Display Number

Stored Page Index

↓

0

Displayed as:

Page 1

because:

page_index + 1

Similarly:

Stored Row Index

↓

0

Displayed as:

Row 1

18. Program 2 Logic

Document

↓

Has "page"?

/ \

Yes No

↓ ↓

+1 Has "row"?

/ \

Yes No

↓ ↓

+1 Not Applicable

19. Main Conclusion

Preserve zero-based indexes internally, but display user-friendly page and row numbers by adding 1.

20. Understanding Python zip()

Before metadata standardisation, we need to understand:

zip()

Definition

zip() pairs corresponding elements from two or more iterables and returns an iterator of tuples.

Example:

```
names = [
    "Ravi",
    "Priya",
    "Rahul"
]



marks = [
    90,
    85,
```

```
    95
]


result = zip(
    names,
    marks
)
```

21. Return Type of zip()

```
print(
    type(result)
)
```

Output:

```
<class 'zip'>
```

So:

zip()

↓

zip object

22. Convert Zip Object to List

```
print(
    list(result)
)
```

Output:

```
[
    ("Ravi", 90),
    ("Priya", 85),
    ("Rahul", 95)
]
```

So:

```
names[0] + marks[0]
      ↓
("Ravi", 90)
names[1] + marks[1]
      ↓
("Priya", 85)
```

23. Easy zip() Memory

List 1:

A B C

List 2:

1 2 3

zip()

↓

(A,1)

(B,2)

(C,3)

Easy definition:

zip() pairs corresponding elements.

24. Program 3 — Metadata Standardisation

Objective

Add common metadata fields to Documents from different source types.

25. Sample Document Collection

```
from pathlib import Path


from langchain_core.documents import (
    Document
)



documents = [

    Document(
        page_content=
            "Students must carry ID cards.",

        metadata={
```

```
        "source":
            "student_rules.txt"
    }
),

Document(
    page_content=
        "Employees receive 12 casual leaves.",

    metadata={
        "source":
            "company_policy.pdf",

        "page":
            0
    }
),

Document(
    page_content=
```

```
        "name: Ravi",

    metadata={
        "source":
            "students.csv",

        "row":
            0
    }
  )

]
```

26. Known Application Information

```
classifications = [

  (
    "txt",
    "Administration"
  ),
```

```
    (
        "pdf",
        "HR"
    ),

    (
        "csv",
        "Admissions"
    )

]
```

Each tuple contains:

source_type

+

department

27. Why Use zip() Here?

We have:

documents

and:

classifications

We want to pair:

Document 1

↔

Classification 1

Document 2

↔

Classification 2

Document 3

↔

Classification 3

So we use:

```
zip(
   documents,
   classifications
)
```

28. Standardising Metadata

```
for document, classification in zip(
    documents,
    classifications
):

    source_type, department = (
        classification
    )


    document.metadata.update({

        "source_type":
            source_type,

        "department":
            department,

        "document_name":
```

```
        Path(
            document.metadata["source"]
        ).name
    })
```

29. Understanding This Line

```
source_type, department = (
    classification
)
```

Suppose:

```
classification = (
    "pdf",
    "HR"
)
```

Then:

```
source_type
    ↓
"pdf"
```

and:

department

↓

"HR"

This is tuple unpacking.

30. Understanding document_name

We create:

```
"document_name":
    Path(
        document.metadata["source"]
    ).name
```

Suppose source is:

C:\data\student_rules.txt

Then:

Path(...).name

returns:

student_rules.txt

31. Standardised Metadata Example

Before:

```
{
   "source":
      "company_policy.pdf",

   "page":
      0
}
```

After:

```
{
   "source":
      "company_policy.pdf",

   "page":
      0,

   "source_type":
      "pdf",
```

```
    "department":
        "HR",

    "document_name":
        "company_policy.pdf"
}
```

Original information is preserved.

New common fields are added.

32. Displaying Standardised Metadata

```
for document in documents:

    metadata = (
        document.metadata
    )



    print(
        "\nNAME:",
        metadata["document_name"]
```

```
)


print(
    "TYPE:",
    metadata["source_type"]
)


print(
    "DEPARTMENT:",
    metadata["department"]
)


print(
    "ORIGINAL SOURCE:",
    metadata["source"]
)
```

33. Program 3 Flow

Different Documents

↓

Different Metadata

↓

zip()

↓

Pair with Classification

↓

metadata.update()

↓

Add Common Fields

↓

Standardised Metadata

34. Main Conclusion

Metadata standardisation gives different Documents a consistent structure without losing their original source-specific information.

35. Program 4 — Multi-Format Document Inspector

Objective

Create Documents from multiple source types:

TXT

PDF

CSV

HTML

Then:

Inspect Metadata

Check Content

Count Documents

Detect Empty Documents

36. Required Installation

```
python -m pip install \
pypdf reportlab beautifulsoup4
```

37. Main Imports

```
import csv

from pathlib import Path

from reportlab.pdfgen import (
    canvas
)

from pypdf import (
    PdfReader
)

from bs4 import (
    BeautifulSoup
)

from langchain_core.documents import (
    Document
)
```

38. Create Project Folder

```
base_dir = (
    Path(__file__).resolve().parent
)



demo_dir = (
    base_dir
    / "module9_final_demo"
)



demo_dir.mkdir(
    exist_ok=True
)
```

Flow:

Python File Folder

↓

module9_final_demo

↓

Store Demo Files

39. Create Text File

```
txt_path = (
    demo_dir
    / "student_rules.txt"
)



txt_path.write_text(
    "Students must carry ID cards.",
    encoding="utf-8"
)
```

40. Create Two-Page PDF

```
pdf_path = (
    demo_dir
    / "company_policy.pdf"
)
```

```
pdf = canvas.Canvas(
    str(pdf_path)
)


pdf.drawString(
    60,
    750,
    "ATTENDANCE POLICY"
)


pdf.drawString(
    60,
    720,
    "Employees must report by 9 AM."
)
```

```
pdf.showPage()


pdf.drawString(
    60,
    750,
    "LEAVE POLICY"
)


pdf.drawString(
    60,
    720,
    "Employees receive 12 casual leaves."
)


pdf.save()
```

This creates:

Page 1

↓

Attendance Policy

Page 2

↓

Leave Policy

41. Create CSV File

```
csv_path = (
    demo_dir
    / "students.csv"
)



csv_path.write_text(
    "name,course\n"
    "Ravi,Python\n"
    "Priya,Java\n",
    encoding="utf-8"
)
```

This contains:

2 Data Rows

So later we expect:

2 CSV Documents

42. Create HTML File

```
html_path = (
    demo_dir
    / "course_page.html"
)



html_path.write_text(
    """
    <html>

    <head>
        <title>
            Course Updates
        </title>
```

```
</head>

<body>

  <main>

    <h1>
      Course Updates
    </h1>

    <p>
      Python batch starts Monday.
    </p>

  </main>

</body>

</html>
"""
```

```
    encoding="utf-8"
)
```

43. Create Document Collection

```
documents = []
```

This one list will contain Documents from:

TXT

PDF

CSV

HTML

That is why this is a:

Multi-Format Document Collection

44. Load Text File

```
documents.append(
    Document(
        page_content=
            txt_path.read_text(
                encoding="utf-8"
            ),
```

```
        metadata={
            "source":
                str(txt_path),

            "source_type":
                "txt",

            "department":
                "Administration"
        }
    )
)
```

The text file creates:

1 Document

45. Load PDF Pages

```
reader = PdfReader(
   str(pdf_path)
)


for page_index, page in enumerate(
   reader.pages
):

   documents.append(
      Document(
         page_content=
            page.extract_text()
            or "",

         metadata={
            "source":
               str(pdf_path),
```

```
            "source_type":
                "pdf",

            "department":
                "HR",

            "page":
                page_index,

            "total_pages":
                len(reader.pages)
        }
    )
)
```

Because there are two PDF pages:

2 Pages

↓

2 Documents

46. Load CSV Rows

```
with csv_path.open(
    "r",
    encoding="utf-8",
    newline=""
) as file:

    reader = csv.DictReader(
        file
    )



    for row_index, row in enumerate(
        reader
    ):

        content = "\n".join(
            f"{key}: {value}"
            for key, value
            in row.items()
```

```
)


documents.append(
    Document(
        page_content=content,

        metadata={
            "source":
                str(csv_path),

            "source_type":
                "csv",

            "department":
                "Admissions",

            "row":
                row_index
        }
```

```
        )
    )
```

Because the CSV has two data rows:

2 Data Rows

↓

2 Documents

47. Load Saved HTML Page

```
soup = BeautifulSoup(
    html_path.read_text(
        encoding="utf-8"
    ),
    "html.parser"
)


main = soup.select_one(
    "main"
)
```

```
documents.append(
    Document(
        page_content=
            main.get_text(
                " ",
                strip=True
            ),

        metadata={
            "source":
                html_path.as_uri(),

            "source_type":
                "html",

            "department":
                "Training",

            "title":
```

```
            soup.title.get_text(
                strip=True
            )
        }
    )
)
```

The HTML file creates:

1 Document

48. Total Documents

We have:

TXT

↓

1 Document

PDF

↓

2 Documents

CSV

↓

2 Documents

HTML

↓

1 Document

Total:

1 + 2 + 2 + 1

↓

6 Documents

49. Multi-Format Document Inspector

Now we inspect all Documents.

```
print(
   "MULTI-FORMAT DOCUMENT INSPECTOR"
)

print(
   "================================"
)
```

```
print(
    "TOTAL DOCUMENTS:",
    len(documents)
)
```

50. Inspect Each Document

```
for number, document in enumerate(
    documents,
    start=1
):

    metadata = (
        document.metadata
    )


    content = " ".join(
        document.page_content.split()
```

```
)


print(
    "\nDOCUMENT",
    number
)


print(
    "TYPE:",
    metadata["source_type"]
)


print(
    "DEPARTMENT:",
    metadata["department"]
)
```

```
    print(
        "CONTENT STATUS:",
        "READY"
        if content
        else "EMPTY"
    )
```

51. Understanding Content Cleaning

```
content = " ".join(
    document.page_content.split()
)
```

This normalises whitespace.

Example:

"Python

  batch   starts"

becomes:

Python batch starts

52. Checking Content Status

"READY"

if content

else "EMPTY"

means:

Content Available?

/ \

Yes No

↓ ↓

READY EMPTY

This is useful for checking whether the loaded Document is usable.

53. Display PDF Page Number

```
if "page" in metadata:


   print(
      "PAGE:",
      metadata["page"] + 1
   )
```

Only PDF Documents normally contain:

page

in this project.

54. Display CSV Row Number

if "row" in metadata:

```
    print(
        "ROW:",
        metadata["row"] + 1
    )
```

Only CSV Documents normally contain:

row

in this project.

55. Display Content Preview

```
print(
    "PREVIEW:",

    content[:70]
    if content
    else "No extractable text"
)
```

This displays only:

First 70 Characters

instead of printing the full content.

56. Document Inspection Flow

Document

↓

Read Metadata

↓

Clean Content

↓

Check Source Type

↓

Check Department

↓

Check Content Status

↓

Show Page / Row if Available

↓

Show Preview

57. Summary by Source Type

We want to know how many Documents came from:

txt

pdf

csv

html

Code:

```
print(
   "\nDOCUMENT COUNT BY TYPE"
)
```

```
print(
    "----------------------"
)


for source_type in [
    "txt",
    "pdf",
    "csv",
    "html"
]:

    count = sum(

        document.metadata[
            "source_type"
        ]
        == source_type

        for document
```

```
    in documents
)



print(
    source_type.upper(),
    ":",
    count
)
```

58. Understanding This sum()

```
count = sum(
    document.metadata[
        "source_type"
    ] == source_type

    for document in documents
)
```

Each comparison produces:

True

or:

False

In Python:

True = 1

False = 0

So sum() counts matching Documents.

Example:

PDF Document → True

PDF Document → True

CSV Document → False

TXT Document → False

sum()

↓

2

59. Expected Count by Type

From our project:

TXT : 1

PDF : 2

CSV : 2

HTML : 1

Total:

6 Documents

60. Count Empty Documents

```
empty_count = sum(

    not document.page_content.strip()

    for document in documents
)
```

Purpose:

Check Every Document

↓

Is Content Empty?

↓

Count Empty Documents

61. Understanding .strip()

document.page_content.strip()

removes surrounding whitespace.

Example:

" "

after .strip() becomes:

""

So:

not document.page_content.strip()

becomes True when the Document contains no meaningful text.

62. Final Inspection Output

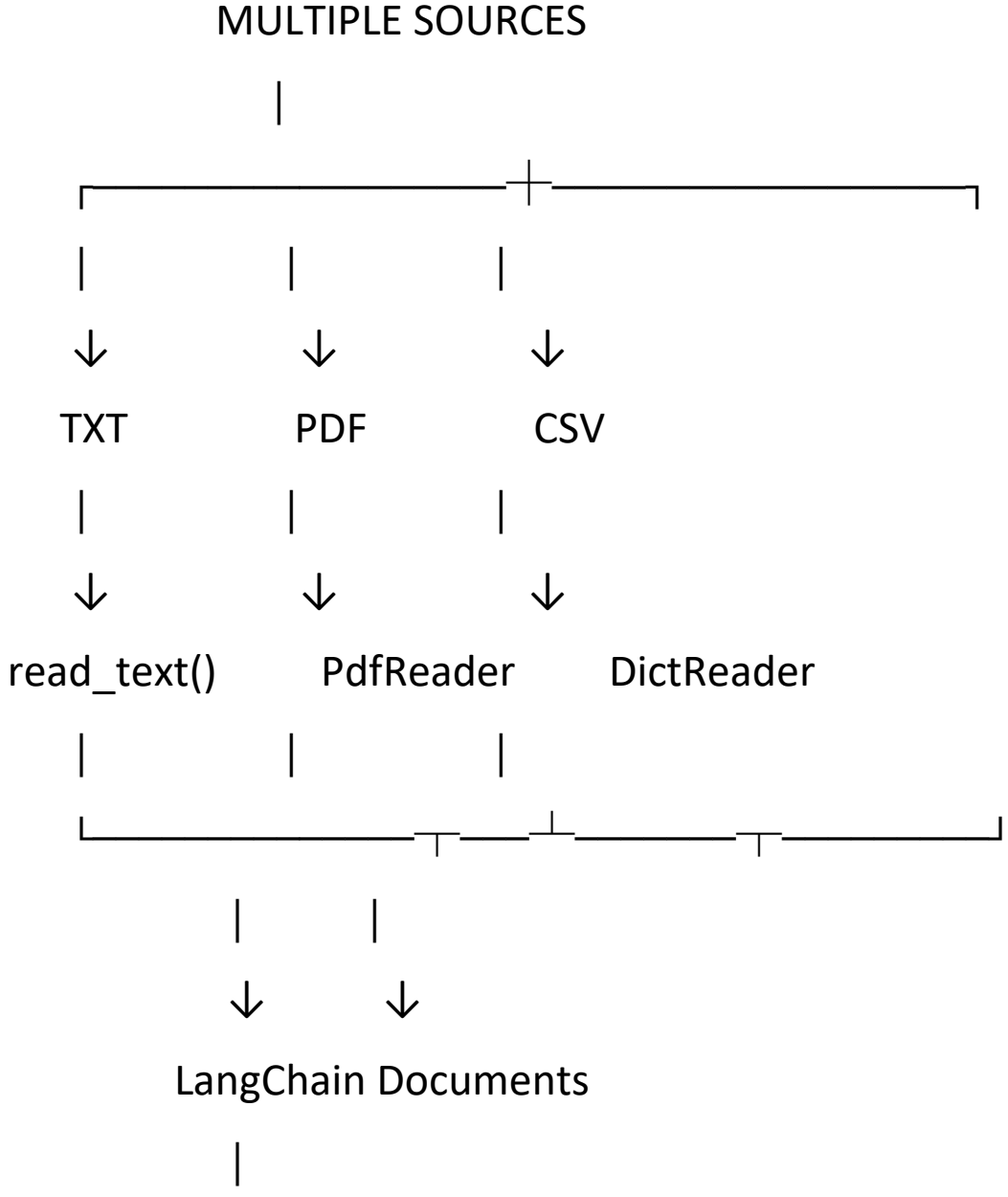
```
print(
    "\nEMPTY DOCUMENTS:",
    empty_count
)

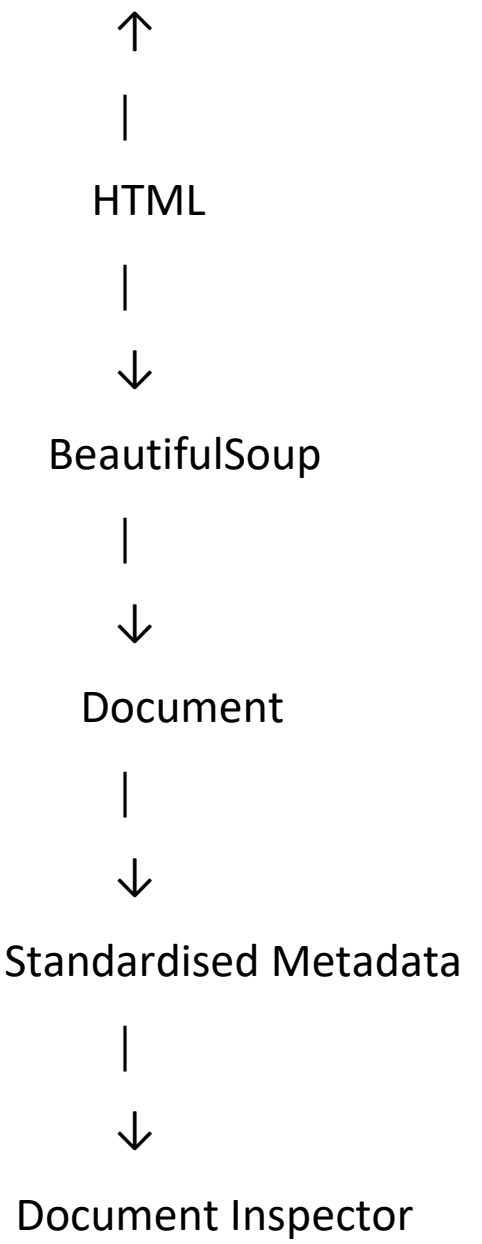

print(
```

```
    "\nINSPECTION COMPLETED"
)
```

This gives us a final quality check.

63. Complete Multi-Format Architecture




HTML
BeautifulSoup
Document
Standardised Metadata
Document Inspector

64. Standard Metadata Structure

A useful common structure may include:

```
{
   "source": "...",
   "source_type": "...",
   "department": "..."
}
```

Source-specific fields may remain.

For PDF:

```
{
   "page": 0,
   "total_pages": 2
}
```

For CSV:

```
{
   "row": 0
}
```

For HTML:

```
{
   "title": "Course Updates"
```

}

65. Common vs Source-Specific Metadata

COMMON FIELDS

---

source

source_type

department

PDF-SPECIFIC

---

page

total_pages

CSV-SPECIFIC

---

row

HTML-SPECIFIC

---

title

Important:

Standardisation does not mean every Document must have exactly the same metadata fields.

It means common fields are consistent while useful source-specific fields are preserved.

66. Why Build a Document Inspector?

Before sending Documents to later stages such as retrieval or other processing, we should verify:

Was content loaded?

Is content empty?

What source type is it?

Which department does it belong to?

Which page or row does it represent?

So:

Load Documents

↓

Inspect Documents

↓

Verify Quality

↓

Use Them Further

67. Important Python Concepts Used

.copy()

Create Independent Metadata Copy

.update()

Add / Update Metadata Fields

.get()

Read Optional Metadata Safely

zip()

Pair Corresponding Items

Tuple Unpacking

```
source_type, department = (
    classification
)
```

enumerate()

Value + Index

sum()

Count Matching Conditions

68. Common Mistake — Replacing Metadata Completely

Risky:

```
document.metadata = {
   "department":
      "HR"
}
```

This may remove:

source

page

row

Better:

```
document.metadata.update({
   "department":
      "HR"
})
```

69. Common Mistake — Converting Stored Page Index to 1

Do not unnecessarily change:

metadata["page"] = 0

into:

metadata["page"] = 1

just for display.

Better:

Store

↓

0

Display

↓

0 + 1 = 1

This preserves the original indexing convention.

70. Common Mistake — Assuming Every Document Has page

Text and HTML Documents may not have:

metadata["page"]

So use conditional logic.

Example:

if "page" in metadata:

71. Common Mistake — Assuming Every Document Has row

Only suitable sources such as CSV may contain:

metadata["row"]

So:

if "row" in metadata:

is safer.

72. Common Mistake — Standardisation Means Removing Special Metadata

No.

Correct idea:

Common Fields

+

Source-Specific Fields

↓

Useful Standardised Metadata

73. Common Mistake — Checking Only Document Count

Suppose:

6 Documents Loaded

That alone does not guarantee the content is useful.

We should also check:

Content Status

Empty Documents

Page / Row

Preview

Source Type

74. Four Main Programs — Revision

Program 1

Enrich Metadata

↓

Preserve Original Source

Program 2

Zero-Based Metadata

↓

Friendly Display Numbers

Program 3

Different Documents

↓

Metadata Standardisation

Program 4

TXT + PDF + CSV + HTML

↓

Document Collection

↓

Document Inspector

75. Today's Master Formula

TXT

PDF

CSV

HTML

↓

LangChain Documents

↓

Consistent Metadata

↓

Content Inspection

↓

Usable Document Collection

76. Final Memory Formula

CONTENT

↓

What?

METADATA

↓

Where / Which / What Type?

STANDARDISATION

↓

Common Structure

INSPECTION

↓

Is the Document Usable?

77. Final One-Line Conclusion

Metadata standardisation helps us organise Documents from TXT, PDF, CSV and HTML sources using common fields while preserving source-specific details, and a Document Inspector helps us verify that every loaded Document contains usable content before further processing.